

PANCHAYAT SEVA AGENT

Complete System Blueprint v2.0

Reactive Agent + Proactive Scheme Notification System

Architecture ReAct Loop	AI Brain Sarvam + Gemini	Tools 9 Agent Tools	New Feature Proactive Alerts	Deployment Hugging Face
----------------------------	-----------------------------	------------------------	---------------------------------	----------------------------

1. Complete System Overview

This blueprint covers the full Panchayat Seva Agent — a two-part system that is both reactive (it helps users who ask questions) and proactive (it alerts users about new government schemes before they even ask). Together, these two parts form a genuinely intelligent, agentic platform — not a chatbot.

✗ Simple Chatbot (What NOT to Build)	☑ True Agent (What This Blueprint Builds)
Hardcoded if/else answers Only handles 6 preset services User must always ask first Cannot handle unknown questions Forgets everything between sessions English only — useless for rural users A 10-line Python script does the same	LLM reasons dynamically every turn Handles ANY Kerala government service Proactively alerts users about new schemes Searches the web for unknown queries Persistent user profiles across sessions Full Malayalam via Sarvam AI Judges will immediately see the difference

1.1 Two Subsystems

SUBSYSTEM 1	Reactive Agent — answers any government service question in Malayalam, using ReAct reasoning and 9 tools. Works 24/7 on demand.
SUBSYSTEM 2	Proactive Notifier — runs every 24 hours, scans for new Kerala government schemes, matches them against user profiles, and sends personalized eligibility alerts in Malayalam.

2. Full System Architecture

The system has eight components. The first six form the reactive agent. The last two form the proactive notification system. All components share the user profile database.

Component	Technology	What It Does
Streamlit UI	Streamlit	Web chat interface — handles Malayalam/English input and streams responses
Conversation Manager	Python Class	Stores full message history and session state across turns
ReAct Agent Core	Python + LLM	Runs the Thought→Action→Observation reasoning loop until Final Answer
LLM Engine	Sarvam AI / Gemini	The brain — understands Malayalam, reasons about problems, calls tools
Tool Executor	Python Functions	Runs the 9 agent tools and returns structured results to the LLM
Knowledge Base	JSON + Web Search	Local service rules + live web search for unknown services
User Profile DB	SQLite / JSON	Persistent user profiles: name, district, income, family size, cards held
Scheme Watcher	APScheduler + Python	Runs every 24 hours — scrapes new scheme announcements from the web
Notification Dispatcher	Python + Streamlit	Matches new schemes to eligible user profiles. Sends in-app Malayalam alerts

2.1 Complete Data Flow

Reactive path:

1. User sends a message → Conversation Manager loads history and user profile from DB
2. Agent Core builds the system prompt and sends full context to LLM
3. LLM generates a Thought and decides on an Action (tool call)
4. Tool Executor runs the tool and returns an Observation to the LLM
5. Steps 3–4 repeat until LLM generates a Final Answer
6. Answer is translated to Malayalam via Sarvam AI, then displayed in Streamlit
7. User profile is updated in the database with any new facts learned

Proactive path (runs every 24 hours independently):

8. Scheme Watcher triggers at 8am — calls web_search for new Kerala scheme announcements
9. Parser extracts: scheme name, eligibility criteria, deadline, application link
10. For each user profile in the database, check_eligibility is called

11. If eligible: a notification record is created in the DB
12. On next Streamlit load, user sees the notification banner in Malayalam

3. The ReAct Reasoning Loop

ReAct (Reason + Act) is the architecture that makes this a real agent. The LLM alternates between Thinking about the problem and Acting by calling tools. This continues until it has enough information to give a complete, verified answer.

T
THOUGHT

User wants to open a tea stall in Thrissur. This is a multi-service workflow. I need: (1) FSSAI food license, (2) Thrissur Corporation trade license, (3) GST check if turnover > Rs.40 lakh. Let me save district=Thrissur to memory and research each service.

A
ACTION

`save_to_memory({"key": "district", "value": "Thrissur"}) web_search({"query": "FSSAI basic registration requirements tea stall Kerala 2025"})`

O
OBSERVATION

FSSAI basic registration: turnover below Rs.12 lakh/year. Apply at foscos.fssai.gov.in. Documents: Aadhaar, address proof, passport photo. Fee: Rs.100/year. Approval: 30 days.

A
ACTION

`find_office({"service": "trade_license", "district": "Thrissur"})`

O
OBSERVATION

Thrissur Corporation, Town Hall Road. Revenue Dept, Counter 3. Phone: 0487-2360028. Hours: Mon–Fri 10am–4pm. Documents: premises proof, Aadhaar, application form.

T
THOUGHT

I have enough for both mandatory licenses. GST is not needed below Rs.40 lakh turnover for a small tea stall. I now have all information needed for a complete Malayalam response.

✓
FINAL
ANSWER

Synthesize both service results into a formatted Malayalam step-by-step action plan with documents, offices, fees, timelines, and a warm closing. Then call `translate_to_malayalam` before sending.

4. All 9 Agent Tools

Tools are the hands of the agent. The LLM autonomously decides which tools to call and in what order. There are 7 reactive tools (used when answering user questions) and 2 proactive tools (used by the Scheme Watcher).

4.1 Reactive Tools (7)



Tool 1 — web_search(query)

Searches the internet for real-time government information. This is the most important tool — it makes the agent capable of answering ANY service query, even ones not in the knowledge base.

INPUT: query: str — search terms in English

ACTION: Calls DuckDuckGo API (no key needed) or Serper API (better quality, free tier). Returns top 5 results.



Tool 2 — get_document_checklist(service, subtype)

Returns the structured document list for a known service from the local knowledge base. Faster and more reliable than web search for the 6 core services.

INPUT: service: str, subtype: str (e.g. 'newborn', 'bpl', 'general')

ACTION: Looks up knowledge.json. Falls back to web_search automatically if service not found.



Tool 3 — find_office(service, district)

Finds the correct government office for a service in the user's district — name, address, phone, counter number, hours.

INPUT: service: str, district: str

ACTION: Queries local office database + web search fallback. Never lets user go to the wrong office.



Tool 4 — check_eligibility(scheme_name, user_profile)

Verifies whether the user qualifies for a government scheme before giving them document requirements. Prevents wasted effort on services they don't qualify for.

INPUT: scheme_name: str, user_profile: dict (income, family_size, category, age, existing_card)

ACTION: Applies eligibility rules from knowledge base + verifies latest criteria via web search.



Tool 5 — translate_to_malayalam(text)

Translates any English text into fluent, natural Malayalam using Sarvam AI's Mayura model. Called as the final step before every Malayalam response.

INPUT: text: str — English content to translate

ACTION: Calls Sarvam AI translate API. Returns fluent Malayalam. Falls back to returning English if API fails.

**Tool 6 — save_to_memory(key, value)**

Saves an important fact about the user to the session memory store so the agent never asks the same question twice.

INPUT: key: str (e.g. 'district'), value: str (e.g. 'Thrissur')

ACTION: Writes to in-memory dict AND to user_profile.json for persistence across sessions.

**Tool 7 — get_from_memory(key)**

Retrieves a previously saved fact about the user. Called at the start of every turn to check what the agent already knows.

INPUT: key: str — the fact to retrieve

ACTION: Reads from memory dict first, then falls back to user_profile.json. Returns 'not found' if missing.

4.2 Proactive Tools (2 — used by Scheme Watcher only)

**Tool 8 — scan_new_schemes()**

Searches the web for Kerala government scheme announcements published in the last 24 hours. The heartbeat of the proactive system.

INPUT: None — runs on a schedule

ACTION: Calls web_search with date-filtered queries. Parses results to extract scheme name, eligibility, deadline, link. Returns list of new schemes.

**Tool 9 — dispatch_notification(user_id, scheme, message)**

Creates a notification record in the database for an eligible user. The notification appears as a banner in the Streamlit UI on their next visit.

INPUT: user_id: str, scheme: dict, message: str (in Malayalam)

ACTION: Writes to notifications table in SQLite. Marks as unread. Streamlit checks for unread notifications on every page load.

5. Complete Project File Structure

Every file has a single responsibility. Build them in the exact order listed.

Project Folder: panchayat-seva-agent/

app.py	← Streamlit UI (entry point)	agent.py	← ReAct agent core & tool		
execution loop	tools.py	← All 9 tool functions	memory.py	← Session memory	
manager	user_profile.py	← Persistent user profile	CRUD	knowledge.json	← Local
service	knowledge base (6 core services)	prompts.py	← System prompt, ReAct format,		
language rules	scheme_watcher.py	← Background scheduler + new scheme scanner			

notifier.py ← Eligibility matcher + notification dispatcher database.py ← SQLite
setup — users, notifications, scheme_log .env ← API keys (NEVER commit to
GitHub) requirements.txt ← All Python dependencies

6. Implementation — File by File

File 1 — requirements.txt

```
streamlit==1.32.0
requests==2.31.0
google-generativeai==0.5.0
python-dotenv==1.0.0
duckduckgo-search==5.3.0
apscheduler==3.10.4
twilio==8.5.0          # optional — for WhatsApp/SMS notifications
```

File 2 — database.py (SQLite Setup)

Three tables: users (profiles), notifications (unread alerts), scheme_log (seen schemes to avoid duplicate alerts).

```
# database.py
import sqlite3, json

def init_db():
    conn = sqlite3.connect('panchayat.db')
    c = conn.cursor()

    # Persistent user profiles
    c.execute('''CREATE TABLE IF NOT EXISTS users (
        user_id TEXT PRIMARY KEY,
        name TEXT, district TEXT, language TEXT,
        income INTEGER, family_size INTEGER,
        category TEXT, age INTEGER,
        existing_cards TEXT,
        notify INTEGER DEFAULT 1,
        created_at TEXT
    ''')

    # Notification queue
    c.execute('''CREATE TABLE IF NOT EXISTS notifications (
        id INTEGER PRIMARY KEY AUTOINCREMENT,
        user_id TEXT, scheme_name TEXT,
        message_ml TEXT,
        is_read INTEGER DEFAULT 0,
        created_at TEXT
    ''')

    # Tracks which schemes have already been processed
    c.execute('''CREATE TABLE IF NOT EXISTS scheme_log (
        scheme_id TEXT PRIMARY KEY,
        scheme_name TEXT,
        found_at TEXT
    ''')
```



```
conn.commit(); conn.close()
```

File 3 — user_profile.py (Profile Manager)

Reads and writes user profiles to SQLite. The agent calls this to persist facts it learns during conversation.

```
# user_profile.py
import sqlite3, json
from datetime import datetime

def save_profile(user_id: str, updates: dict):
    conn = sqlite3.connect('panchayat.db')
    c = conn.cursor()
    for key, val in updates.items():
        c.execute(f'UPDATE users SET {key}=? WHERE user_id=?', (val,
user_id))
    conn.commit(); conn.close()

def load_profile(user_id: str) -> dict:
    conn = sqlite3.connect('panchayat.db')
    c = conn.cursor()
    row = c.execute('SELECT * FROM users WHERE user_id=?',
(user_id,)).fetchone()
    conn.close()
    if not row: return {}
    cols = ['user_id', 'name', 'district', 'language', 'income',
'family_size', 'category', 'age', 'existing_cards', 'notify', 'created_at']
    return dict(zip(cols, row))

def create_user(user_id: str, language: str = 'ml'):
    conn = sqlite3.connect('panchayat.db')
    conn.execute('INSERT OR IGNORE INTO users (user_id, language,
created_at) VALUES (?, ?, ?)',
                (user_id, language, datetime.now().isoformat()))
    conn.commit(); conn.close()
```

File 4 — tools.py (All 9 Tools)

Tools 1 & 2 — web_search and get_document_checklist

```
# tools.py
import os, json, requests
from duckduckgo_search import DDGS

def web_search(query: str) -> str:
    try:
```

```

        with DDGS() as ddgs:
            results = list(ddgs.text(query, max_results=5))
            if not results: return 'No results found.'
            return '\n\n'.join([f"Title: {r['title']}\nInfo: {r['body']}" for r
in results])
        except Exception as e:
            return f'Search failed: {str(e)}'

def get_document_checklist(service: str, subtype: str = 'general') -> str:
    with open('knowledge.json') as f: KB = json.load(f)
    if service not in KB:
        return web_search(f'{service} documents required Kerala panchayat
2025')
    svc = KB[service]
    sub = svc['subtypes'].get(subtype, list(svc['subtypes'].values())[0])
    return (f"Documents: {'', '.join(sub['documents'])}. "
            f"Office: {sub['office']}. Fee: {sub['fee']}. "
            f"Timeline: {sub['timeline']}. Online:
{sub.get('online', 'N/A')}")

```

Tools 3 & 4 — find_office and check_eligibility

```

def find_office(service: str, district: str) -> str:
    return web_search(f'{service} office {district} Kerala address phone
hours 2025')

def check_eligibility(scheme_name: str, user_profile: dict) -> str:
    profile_str = json.dumps(user_profile)
    return web_search(f'{scheme_name} eligibility Kerala criteria
{profile_str}')

```

Tool 5 — translate_to_malayalam

```

def translate_to_malayalam(text: str) -> str:
    url = 'https://api.sarvam.ai/translate'
    headers = {'API-Subscription-Key': os.getenv('SARVAM_API_KEY'),
               'Content-Type': 'application/json'}
    payload = {'input': text, 'source_language_code': 'en-IN',
               'target_language_code': 'ml-IN', 'model': 'mayura:v1',
               'enable_preprocessing': True}

    try:
        r = requests.post(url, headers=headers, json=payload, timeout=10)
        return r.json().get('translated_text', text)
    except:
        return text # graceful fallback to English

```

Tools 6 & 7 — Memory

```

_memory = {}

def save_to_memory(key: str, value: str) -> str:
    _memory[key] = value
    return f'Saved: {key} = {value}'

```

```
def get_from_memory(key: str) -> str:
    return _memory.get(key, 'Not found')
```

Tools 8 & 9 — Proactive Tools

```
from datetime import datetime
import sqlite3

def scan_new_schemes() -> list:
    # Search for schemes announced in last 24 hours
    queries = [
        'new Kerala government scheme announced today 2025',
        'Kerala welfare scheme launched this week',
        'Kerala panchayat benefit scheme new announcement'
    ]
    schemes = []
    for q in queries:
        result = web_search(q)
        schemes.append({'raw': result, 'found_at':
datetime.now().isoformat()})
    return schemes

def dispatch_notification(user_id: str, scheme_name: str, message_ml: str) -
> str:
    conn = sqlite3.connect('panchayat.db')
    conn.execute(
        'INSERT INTO notifications (user_id, scheme_name, message_ml,
created_at) VALUES (?, ?, ?, ?)',
        (user_id, scheme_name, message_ml, datetime.now().isoformat())
    )
    conn.commit(); conn.close()
    return f'Notification queued for user {user_id}'
```

File 5 — agent.py (ReAct Loop)

The orchestration engine. Sends conversation to the LLM, parses Thought/Action/Observation, executes tools, loops until Final Answer.

```
# agent.py
import re, json, os
import google.generativeai as genai
from tools import (web_search, get_document_checklist, find_office,
                    check_eligibility, translate_to_malayalam,
                    save_to_memory, get_from_memory)
from prompts import build_system_prompt, detect_query_type

TOOLS = {
    'web_search': lambda q: web_search(q),
    'get_document_checklist': lambda s, t='general':
get_document_checklist(s, t),
    'find_office': lambda s, d: find_office(s, d),
```

```

    'check_eligibility': lambda s, p: check_eligibility(s, p),
    'translate_to_malayalam': lambda t: translate_to_malayalam(t),
    'save_to_memory': lambda k, v: save_to_memory(k, v),
    'get_from_memory': lambda k: get_from_memory(k),
}

genai.configure(api_key=os.getenv('GEMINI_API_KEY'))

def run_agent(user_message: str, history: list) -> str:
    qtype = detect_query_type(user_message)
    system = build_system_prompt(**qtype)
    model = genai.GenerativeModel('gemini-2.0-flash',
                                   system_instruction=system)
    messages = history + [{'role': 'user', 'parts': [user_message]}]

    for _ in range(10): # max 10 reasoning steps
        resp = model.generate_content(messages)
        text = resp.text

        # Parse Action call
        match = re.search(r'Action:\s*(\w+)\s*((.+?)\s*)', text, re.DOTALL)
        if match:
            tool_name = match.group(1)
            try:
                args = json.loads('{ ' + match.group(2) + ' }')
            except:
                args = json.loads(match.group(2))
            if tool_name in TOOLS:
                obs = str(TOOLS[tool_name](**args))
                messages.append({'role': 'model', 'parts': [text]})
                messages.append({'role': 'user', 'parts': [f'Observation:
{obs}']}])
                continue

        # No Action → extract Final Answer
        final = re.search(r'Final Answer:\s*(.+)', text, re.DOTALL)
        return final.group(1).strip() if final else text.strip()

    return 'Unable to complete. Please try again.'

```

File 6 — scheme_watcher.py (The Proactive Watcher)

This is the heart of the new proactive system. It runs automatically every 24 hours using APScheduler, scans for new government scheme announcements, and triggers the notification dispatcher for eligible users.

```

# scheme_watcher.py
import os, json, sqlite3, hashlib
from datetime import datetime
from apscheduler.schedulers.background import BackgroundScheduler
from tools import web_search, check_eligibility, translate_to_malayalam
from notifier import notify_eligible_users

```

```

from database import init_db

def extract_scheme_info(raw_text: str) -> list:
    '''Uses Gemini to parse raw search results into structured scheme
    data'''
    import google.generativeai as genai
    genai.configure(api_key=os.getenv('GEMINI_API_KEY'))
    model = genai.GenerativeModel('gemini-2.0-flash')
    prompt = f'''
    Extract all government scheme announcements from this text.
    Return a JSON array. Each item must have:
    - name: scheme name
    - eligibility: who qualifies (income, age, category, etc.)
    - deadline: application deadline if mentioned
    - link: official URL if present
    - summary: one sentence description
    Text: {raw_text}
    Return only valid JSON, nothing else.
    '''
    resp = model.generate_content(prompt)
    try:
        return json.loads(resp.text.strip().strip('```json').strip('```'))
    except:
        return []

def is_new_scheme(scheme_name: str) -> bool:
    '''Returns True if this scheme has not been seen before'''
    scheme_id = hashlib.md5(scheme_name.encode()).hexdigest()
    conn = sqlite3.connect('panchayat.db')
    exists = conn.execute(
        'SELECT 1 FROM scheme_log WHERE scheme_id=?',
        (scheme_id,)).fetchone()
    if not exists:
        conn.execute(
            'INSERT INTO scheme_log VALUES (?, ?, ?)',
            (scheme_id, scheme_name, datetime.now().isoformat()))
        conn.commit()
    conn.close()
    return not exists

def watch_for_new_schemes():
    '''Main watcher function - runs every 24 hours'''
    print(f'[{datetime.now()}] Scheme Watcher started...')
    queries = [
        'Kerala government new scheme announced 2025',
        'Kerala welfare benefit launched this week',
        'Kerala panchayat housing scholarship scheme new'
    ]
    for query in queries:
        raw = web_search(query)
        schemes = extract_scheme_info(raw)
        for scheme in schemes:
            if is_new_scheme(scheme.get('name', '')):
                print(f"New scheme found: {scheme['name']}")

```

```

        notify_eligible_users(scheme)

def start_scheduler():
    '''Start the background scheduler - call this in app.py'''
    scheduler = BackgroundScheduler()
    scheduler.add_job(
        watch_for_new_schemes,
        'cron',
        hour=8,          # Runs every day at 8am
        minute=0
    )
    scheduler.start()
    return scheduler

```

File 7 — notifier.py (Eligibility Matcher + Dispatcher)

Loads all user profiles from the database, checks each one for eligibility against the new scheme, and dispatches personalized Malayalam notifications to those who qualify.

```

# notifier.py
import sqlite3
from tools import check_eligibility, translate_to_malayalam,
dispatch_notification

def get_all_users() -> list:
    conn = sqlite3.connect('panchayat.db')
    rows = conn.execute(
        'SELECT user_id, name, district, income, family_size, category, age,
notify'
        ' FROM users WHERE notify=1').fetchall()
    conn.close()
    cols =
['user_id', 'name', 'district', 'income', 'family_size', 'category', 'age', 'notify
']
    return [dict(zip(cols, row)) for row in rows]

def build_notification_message(user: dict, scheme: dict) -> str:
    '''Build a personalised English message, then translate to Malayalam'''
    name      = user.get('name', 'valued citizen')
    district  = user.get('district', 'Kerala')
    msg_en = (
        f"Hello {name}! A new government scheme has been announced that you
may be eligible for.\n"
        f"Scheme: {scheme['name']}\n"
        f"Summary: {scheme.get('summary', 'New benefit scheme')}\n"
        f"Deadline: {scheme.get('deadline', 'Check official portal')}\n"
        f"Apply here: {scheme.get('link', 'https://kerala.gov.in')}\n"
        f"Contact your nearest Akshaya centre in {district} for help
applying."
    )
    return translate_to_malayalam(msg_en)

def notify_eligible_users(scheme: dict):

```

```

'''Check all users and notify the eligible ones'''
users = get_all_users()
for user in users:
    profile = {
        'income':      user.get('income', 0),
        'family_size': user.get('family_size', 0),
        'category':     user.get('category', 'general'),
        'age':          user.get('age', 0)
    }
    eligibility_result = check_eligibility(scheme['name'], profile)
    # If the eligibility check mentions likely/eligible/qualifies →
    notify
    keywords = ['eligible', 'qualifies', 'likely', 'meets', 'can apply']
    if any(k in eligibility_result.lower() for k in keywords):
        msg_ml = build_notification_message(user, scheme)
        dispatch_notification(user['user_id'], scheme['name'], msg_ml)
        print(f"Notified user {user['user_id']} about {scheme['name']}")

```

File 8 — app.py (Streamlit UI with Notifications)

The main entry point. Initializes the database, starts the scheduler, shows unread notifications, and runs the chat interface.

```

# app.py
import streamlit as st
import sqlite3
from agent import run_agent
from database import init_db
from scheme_watcher import start_scheduler
from user_profile import create_user, load_profile

# One-time initialization
init_db()

st.set_page_config(page_title='Panchayat Seva Agent', page_icon='🏠',
layout='wide')

# Start background scheduler (only once per session)
if 'scheduler_started' not in st.session_state:
    start_scheduler()
    st.session_state.scheduler_started = True

# Session user ID
if 'user_id' not in st.session_state:
    import uuid
    st.session_state.user_id = str(uuid.uuid4())
    create_user(st.session_state.user_id)

# Show unread notifications as banners
def show_notifications(user_id):
    conn = sqlite3.connect('panchayat.db')
    rows = conn.execute(

```

```

        'SELECT id, scheme_name, message_ml FROM notifications'
        ' WHERE user_id=? AND is_read=0', (user_id,)).fetchall()
    for row in rows:
        st.info(f'🔔 **{row[1]}**\n\n{row[2]}')
        conn.execute('UPDATE notifications SET is_read=1 WHERE id=?',
            (row[0],))
        conn.commit(); conn.close()

# — UI Layout —————
st.title('🏠 Panchayat Seva Agent')
st.caption('Kerala Government Services Assistant – Malayalam & English')

show_notifications(st.session_state.user_id)

if 'messages' not in st.session_state: st.session_state.messages = []
if 'history' not in st.session_state: st.session_state.history = []

for msg in st.session_state.messages:
    with st.chat_message(msg['role']): st.write(msg['content'])

if prompt := st.chat_input('Type in Malayalam or English... / മലയാളത്തിൽ  

    ടൈപ്പ് ചെയ്യൂ...'):
    st.session_state.messages.append({'role': 'user', 'content': prompt})
    with st.chat_message('user'): st.write(prompt)
    with st.chat_message('assistant'):
        with st.spinner('Thinking... / ചിന്തിക്കുന്നു...'):
            reply = run_agent(prompt, st.session_state.history)
        st.write(reply)
    st.session_state.messages.append({'role': 'assistant', 'content':
reply})
    st.session_state.history += [{'role': 'user', 'parts': [prompt]},
{'role': 'model', 'parts': [reply]}]

```


7. Full Demo Conversation — Reactive + Proactive

This shows both systems in action in a single session. The proactive notification appears as a banner when the user loads the app. The reactive agent then handles a multi-service question.

ALERT	[Notification banner appears when Rajan opens the app] 📢 PM Awas Yojana — Gramin Rajan, PM Awas Yojana-yil ninnum veedu labhikkann ninnalk yogyatha undennu thonunnu. Last date: March 31. Apply: pmayg.nic.in. Akshaya centre-il poyi sahaayam thedan. (You may be eligible for PM Awas Yojana housing scheme. Deadline: March 31.)
USER	Namaskaram! Njan Rajan, Mukundapuram. Ente veedu puchha aanu, PM Awas scheme-il eligibility und? Koodathe oru chaya kada thuranganam. Enthu cheyyanam?
THOUGHT	User is Rajan from Mukundapuram (Thrissur district). Two requests: (1) check PM Awas Yojana eligibility, (2) tea stall opening workflow. Let me save his name and district first, then check eligibility, then research tea stall licenses.
ACTION	save_to_memory({"key": "name", "value": "Rajan"}) save_to_memory({"key": "district", "value": "Thrissur"}) check_eligibility({"scheme_name": "PM Awas Yojana Gramin", "user_profile": {"income": 0, "district": "Thrissur", "has_pucca_house": false}})
OBSERVATION	PM Awas Yojana Gramin: Eligible if no pucca house, income below Rs.3 lakh/year (EWS) or Rs.6 lakh (LIG). Must be on SECC 2011 beneficiary list or state BPL list. Apply through Gram Panchayat. Documents: Aadhaar, income cert, bank passbook, land document.
ACTION	web_search({"query": "FSSAI basic registration Kerala tea stall 2025 documents fee"}) find_office({"service": "trade_license", "district": "Thrissur"})
OBSERVATION	FSSAI: Basic registration for <Rs.12L turnover. Fee Rs.100/year. Apply: foscoss.fssai.gov.in. Docs: Aadhaar, address proof, photo. Trade License: Thrissur Corporation, Town Hall Road, Counter 3. Phone: 0487-2360028. Hours: Mon-Fri 10am-4pm.
AGENT ✓	Rajan, ഈ 2 കാര്യങ്ങൾക്ക് ഉത്തരം ഇതാ: 🏠 PM Awas Yojana — നിങ്ങൾക്ക് ഇതിൽ apply ചെയ്യാം. Documents: Aadhaar, Income cert, Bank passbook, Land doc. Mukundapuram Panchayat Office-ൽ apply ചെയ്യൂ. Deadline: March 31! 🕒 ചായക്കടക്ക് 2 License വേണം: [1] FSSAI — foscoss.fssai.gov.in — Rs.100/year — Docs: Aadhaar, address proof [2] Trade License — Thrissur Corporation, Town Hall Road, Counter 3 — 0487-2360028 ആദ്യം PM Awas apply ചെയ്യൂ (deadline March 31!), പിന്നെ licenses. All the best Rajan! 🙏

Notice: The agent handled both eligibility screening AND a multi-service business workflow in a single conversation. It remembered Rajan's district automatically. Both subsystems (proactive alert + reactive agent) worked together seamlessly. This is the demo to show on Demo Day.

8. Four-Week Build Roadmap

Week	Theme	What to Build
Week 1 Mar 8–14	Foundation	Get all API keys. Install dependencies. Build database.py + user_profile.py. Write knowledge.json for 6 core services. Test web_search and translate_to_malayalam manually.
Week 2 Mar 15–22	Reactive Agent	Write prompts.py, tools.py (7 reactive tools), agent.py (ReAct loop). Connect Streamlit UI. Test 15 different user queries end-to-end including Malayalam input.
Week 3 Mar 23–31	Proactive System	Build scheme_watcher.py with APScheduler. Build notifier.py with eligibility matching. Test full notification flow: new scheme → DB → user alert. Add voice input with Sarvam speech-to-text.
Week 4 Apr 1–5	Polish + Deploy	Deploy to Hugging Face Spaces. Test on mobile. Record 3-min demo video showing both systems. Build 5-slide deck. Practice live demo with a real Malayalam speaker.

9. Demo Day Strategy

9.1 Your 3-Minute Demo Script

13. **0:00–0:30 The Hook** — Open with a real story: 'A family in Mukundapuram qualified for a housing scheme last year. They found out 3 months after the deadline. This agent would have notified them the day it was announced.' Then open the live demo.
14. **0:30–1:30 Proactive Demo** — Show a notification banner appearing when you open the app. Say: 'The agent ran at 8am today, scanned for new schemes, found this one, matched it to our user's profile, and sent this alert — automatically, in Malayalam, without the user asking.' Let that land.
15. **1:30–2:30 Reactive Demo** — Have a real person type a question in Malayalam. Show the Thought→Action→Observation loop running. Show the agent handling something it was never explicitly programmed for by searching the web.
16. **2:30–3:00 The Scale Argument** — 'Kerala has 941 Gram Panchayats. 3.5 crore citizens. Hundreds of schemes a year. This agent can serve all of them, in Malayalam, for free, using Sarvam AI's Sovereign model. It doesn't just answer questions. It makes sure nobody misses a benefit they deserve.'

9.2 5 Slides

- Slide 1 — Problem: One image of a rural family. Three lines. No jargon.
- Slide 2 — Reactive Agent: Screenshot of Malayalam conversation with multi-service answer.
- Slide 3 — Proactive System: Diagram showing Watcher → Matcher → Notification flow.

- Slide 4 — Architecture: ReAct loop + 9 tools + SQLite. Shows technical depth.
- Slide 5 — Scale: 941 Panchayats. 10+ Indian languages. Deployable on any server.

The Single Thing That Will Win

Bring a real person — elderly, speaks only Malayalam — and let them use the agent live. Then show the proactive notification appearing on its own. When a judge watches real human impact happening in real time, in Malayalam, automatically — no competitor can beat that moment. That is Sovereign AI. That is what AI Kyro is looking for.

Built with Sarvam AI. Designed for Kerala. Powered by Sovereign AI.

AI Kyro Capstone 2025 • Thrissur, Kerala • Version 2.0